MnemoCalendar — Convention & Development Guide

> **Mục tiêu:** thống nhất cách tổ chức mã nguồn, đặt tên, dùng MVVM Toolkit, truy cập SQLite và phối hợp Git cho đồ án WPF/C# của nhóm 2–5 người.
>
> **Stack đã chốt:** WPF + C# · `Microsoft.Data.Sqlite` (không dùng EF Core) · `CommunityToolkit.Mvvm` · WPF-UI (`lepoco/WPF-UI`).

---

1. Nguyên tắc làm việc

1. **Ưu tiên MVP trước:** tạo/xem/sửa/xóa sự kiện, lịch tháng/tuần, quản lý tag, lọc `tagA` / `tagB` / chưa có tag và tìm kiếm. Recurrence, reminder, đồng bộ cloud và all-day event chỉ làm khi MVP đã ổn.
2. **View chỉ lo hiển thị:** không viết SQL, không xử lý nghiệp vụ trong `.xaml.cs`.
3. **ViewModel giữ UI state và command:** dùng `ObservableProperty`, `RelayCommand`, `ObservableCollection` khi phù hợp.
4. **Repository là nơi chứa SQL:** chỉ Repository làm việc trực tiếp với `Microsoft.Data.Sqlite`.
5. **Mọi thay đổi đi qua PR:** không push trực tiếp vào `main`; PR phải có người review và chạy kiểm tra trước khi merge.
6. **Một task có một owner chính:** người khác có thể hỗ trợ/review, nhưng cần thống nhất trước khi cùng sửa một file.

---

2. Kiến trúc và cấu trúc thư mục

Đặt `MainWindow.xaml` ở **thư mục gốc của project WPF**, không nhất thiết ở gốc toàn bộ repository. Với một project duy nhất, đề xuất cấu trúc:

```
MnemoCalendar/
├── MnemoCalendar.sln
├── README.md
├── CONTRIBUTING.md
├── .gitignore
│
├── docs/                              # Tài liệu đồ án, thiết kế và demo
│   ├── requirements/
│   │   └── functional-requirements.md # Functional / non-functional requirements
│   ├── design/
│   │   ├── figma-link.md              # Link Figma và ghi chú thiết kế
│   │   └── user-flow.png              # Luồng thao tác chính
│   ├── database/
│   │   ├── schema.sql                 # DDL tạo database
│   │   └── erd.png                    # Sơ đồ quan hệ bảng
│   └── demo/
│       └── demo-script.md             # Kịch bản demo và Q&A
│
├──── MnemoCalendar/                   # Thư mục gốc của project WPF
│       ├── MnemoCalendar.csproj
│       ├── App.xaml                   # Application resources / theme
│       ├── App.xaml.cs                # Khởi tạo ứng dụng, composition root
│       ├── MainWindow.xaml            # Cửa sổ chính / app shell
│       ├── MainWindow.xaml.cs         # Chỉ xử lý việc riêng của Window
│       │
│       ├── Models/                    # Model dữ liệu thuần C#
│       │   ├── CalendarEvent.cs
│       │   ├── Tag.cs
│       │   └── EventFilter.cs
│       │
│       ├── Views/                     # XAML giao diện
│       │   ├── CalendarView.xaml
│       │   ├── CalendarView.xaml.cs
│       │   ├── EventEditorDialog.xaml
│       │   ├── EventEditorDialog.xaml.cs
│       │   ├── TagManagerView.xaml
│       │   ├── TagManagerView.xaml.cs
│       │   └── Controls/
│       │       └── EventCard.xaml
│       │
│       ├── ViewModels/                # State + command cho từng View
│       │   ├── MainViewModel.cs
│       │   ├── CalendarViewModel.cs
│       │   ├── EventEditorViewModel.cs
│       │   └── TagManagerViewModel.cs
│       │
│       ├── Data/                      # Hạ tầng SQLite
│       │   ├── SqliteConnectionFactory.cs
│       │   ├── DatabaseInitializer.cs
│       │   └── schema.sql
│       │
│       ├── Repositories/              # SQL queries và mapping DB ↔ Model
│       │   ├── IEventRepository.cs
│       │   ├── SqliteEventRepository.cs
```

```
│       │   ├── ITagRepository.cs
│       │   └── SqliteTagRepository.cs
│       │
│       ├── Services/                 # Nghiệp vụ dùng chung / điều phối
│       │   ├── CalendarService.cs
│       │   ├── SearchService.cs
│       │   └── DialogService.cs
│       │
│       ├── Resources/                # Tài nguyên giao diện dùng chung
│       │   ├── Styles/
│       │   │   ├── Colors.xaml
│       │   │   ├── Buttons.xaml
│       │   │   └── Calendar.xaml
│       │   ├── Templates/
│       │   │   ├── EventTemplates.xaml
│       │   │   └── TagTemplates.xaml
│       │   └── Assets/
│       │       ├── Icons/
│       │       └── Images/
│       │
│       ├── Converters/               # IValueConverter nếu thực sự cần
│       │   └── DateTimeToDisplayConverter.cs
│       │
│       ├── Behaviors/                # Behavior tái sử dụng, nếu cần
│       │   └── EventDragBehavior.cs
│       │
│       └── Helpers/                  # Hàm tiện ích thuần, không chứa nghiệp vụ lớn
│           └── DateTimeHelper.cs
│
└── tests/                            # Tùy thời gian: project test riêng
    └── MnemoCalendar.Tests/
```

Vai trò từng lớp

```
View (XAML)
   │  Binding / Command
   ▼
ViewModel
   │  gọi method nghiệp vụ
   ▼
Service (khi cần điều phối nhiều bước)
   │
   ▼
Repository
   │  SQL parameterized + mapping
   ▼
SQLite database
```

- **Models:** biểu diễn dữ liệu, ví dụ `CalendarEvent`, `Tag`, `EventFilter`. Model không tham chiếu `TextBox`, `Window` hay control WPF.

- **Views:** XAML và code-behind tương ứng. Code-behind chỉ dành cho thao tác gắn chặt với view (ví dụ `InitializeComponent`, xử lý focus đặc biệt); không đặt SQL hoặc logic CRUD ở đây.
- **ViewModels:** property được bind lên UI và command mà UI gọi. ViewModel không trực tiếp tạo `SqliteConnection`.
- **Data:** tạo connection, khởi tạo schema và cấu hình database.
- **Repositories:** câu SQL, parameters, transaction và chuyển dữ liệu reader thành Model.
- **Services:** điều phối nghiệp vụ nếu một thao tác cần gọi nhiều Repository hoặc có quy tắc dùng chung.
- **Resources:** style, template, màu, icon, ảnh. `Resources` là tên nên dùng thay vì `scr` hoặc một tên viết tắt khó hiểu.

Lưu ý quan trọng: `CalendarViewModel.cs` là file C# thông thường, **không có** `CalendarViewModel.xaml` hoặc `.xaml.cs`. Chỉ View mới có cặp `.xaml` / `.xaml.cs`. Đặt tên ViewModel theo View tương ứng để dễ tìm, nhưng ViewModel không phải một file XAML.

Với đồ án nhỏ, không cần tạo tất cả folder/class ngay từ đầu. Tạo folder khi có file thật; tránh abstraction hoặc interface chỉ để "đủ kiến trúc" mà không đem lại ích lợi.

---

3. Quy tắc đặt tên

Dùng quy ước .NET/C# phổ biến và giữ thống nhất trong toàn project.

Thành phần	Quy tắc	Ví dụ
Namespace	PascalCase theo folder	`MnemoCalendar.ViewModels`
Class / record	PascalCase, danh từ	`CalendarEvent`, `EventFilter`
Interface	`I` + PascalCase	`IEventRepository`
Public property	PascalCase	`SelectedDate`, `Events`
Private field	`_camelCase`	`_selectedDate`, `_eventRepository`
Method	PascalCase, động từ + đối tượng	`LoadEvents`, `SaveEvent`
Parameter / local variable	camelCase	`eventId`, `startAt`
Constant	PascalCase	`DefaultEventDurationMinutes`

Thành phần	Quy tắc	Ví dụ
Enum / enum member	PascalCase	`CalendarViewMode.Month`
Command method	động từ + đối tượng, không thêm `Command` vào method	`SaveEvent()` → `SaveEventCommand`
XAML file	PascalCase, khớp tên class	`CalendarView.xaml`
Resource key	PascalCase có ý nghĩa	`PrimaryButtonStyle`
SQL table / column	`snake_case`, chữ thường	`events`, `start_at_utc`

Quy tắc riêng cho CommunityToolkit.Mvvm

- ViewModel dùng `partial class`.
- Kế thừa `ObservableObject` (hoặc `ObservableValidator` nếu cần validation tích hợp).
- Dùng field private dạng `_camelCase` với `[ObservableProperty]`; Toolkit sinh property PascalCase.
- Dùng `[RelayCommand]` trên method; Toolkit sinh property command có hậu tố `Command`.
- Không tự viết lại `INotifyPropertyChanged` nếu đã dùng `ObservableObject` / `[ObservableProperty]`.
- Chỉ dùng `ObservableCollection<T>` cho danh sách cần thông báo khi thêm/xóa item. Nếu thay toàn bộ danh sách, có thể gán một collection mới qua property có notification.

Ví dụ:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using System.Collections.ObjectModel;

namespace MnemoCalendar.ViewModels;

public partial class EventEditorViewModel : ObservableObject
{
    [ObservableProperty]
    private string _title = string.Empty;

    [ObservableProperty]
    private DateTimeOffset _startAt = DateTimeOffset.Now;

    [ObservableProperty]
    private DateTimeOffset _endAt = DateTimeOffset.Now.AddHours(1);

    [ObservableProperty]
    private ObservableCollection<Tag> _availableTags = new();

    [RelayCommand]
    private void SaveEvent()
    {
        // Validate → gọi service/repository → báo kết quả cho View.
    }
}
```

`[ObservableProperty]` sẽ tạo property như `Title`, `StartAt`, `EndAt`; `[RelayCommand]` trên `SaveEvent()` sẽ tạo command để View bind bằng `SaveEventCommand`.

Ví dụ binding trong XAML:

```
<TextBox
    Text="{Binding Title, Mode=TwoWay,
                   UpdateSourceTrigger=PropertyChanged}" />

<Button
    Content="Lưu"
    Command="{Binding SaveEventCommand}" />
```

Quy tắc Binding và Command

- Binding tới **property**, không bind trực tiếp hai control với nhau nếu dữ liệu đó thuộc state của ViewModel. Ví dụ `TextBox.Text` và `TextBlock.Text` cùng bind tới `Name`.
- Dùng `Mode=TwoWay` cho input cần ghi ngược về ViewModel.
- Chỉ dùng `UpdateSourceTrigger=PropertyChanged` khi cần cập nhật source ngay trong lúc gõ; không phải control nào cũng cần.
- Nút bấm gọi `ICommand` / `RelayCommand`, không gắn business logic vào `Click` trong code-behind.

- Khi command chỉ được thực thi khi dữ liệu hợp lệ, dùng `CanExecute` hoặc validation và yêu cầu command cập nhật trạng thái khi property liên quan đổi.
- Code-behind không được gọi SQL hoặc tự sửa collection nghiệp vụ.

Quy tắc bất đồng bộ

- Chỉ đặt hậu tố `Async` cho method thực sự có semantics bất đồng bộ.
- Không dùng `async void` cho method nghiệp vụ; dùng `Task` / `Task<T>`.
- Không gọi `.Result` hoặc `.Wait()` trên UI thread vì có thể làm giao diện đứng hoặc deadlock.
- Với `Microsoft.Data.Sqlite`, xem lưu ý đa luồng ở mục 5: các API ADO.NET `Async` của provider vẫn thực thi đồng bộ do SQLite không hỗ trợ async I/O.

---

4. Quy ước Model và Functional scope

4.1. Event fields trong MVP

Một `CalendarEvent` nên có tối thiểu:

Property	Kiểu gợi ý	Bắt buộc?	Ghi chú
`Id`	`long`	Có sau khi lưu	Khóa chính SQLite
`Title`	`string`	Có	Không được rỗng sau khi trim
`StartAt`	`DateTimeOffset`	Có	Thời điểm bắt đầu
`EndAt`	`DateTimeOffset`	Có	Phải sau `StartAt`
`Location`	`string?`	Không	Địa điểm
`Description`	`string?`	Không	Mô tả
`Tags`	collection of `Tag`	Không	0..n tag
`ColorHex`	`string?`	Không	Màu riêng cho event, nếu nhóm làm trong MVP
`CreatedAtUtc`	`DateTimeOffset`	Có	Thời điểm tạo

Property	Kiểu gợi ý	Bắt buộc?	Ghi chú
`UpdatedAtUtc`	`DateTimeOffset`	Có	Thời điểm cập nhật cuối

Tag semantics đề xuất:

- Một event có thể không có tag, có một tag hoặc nhiều tag.
- Tag được gắn qua bảng liên kết nhiều-nhiều `event_tags`.
- `Untagged` nghĩa là event **không có bất kỳ bản ghi liên kết nào** trong `event_tags`; không tạo một tag tên `"Chưa có tag"` để giả lập.
- Khi lọc nhiều tag, đề xuất dùng semantics **OR**: event khớp nếu có ít nhất một tag đã chọn.
- Filter `Chưa có tag` nên là một lựa chọn riêng, không kết hợp đồng thời với danh sách tag đã chọn để tránh kết quả khó hiểu.
- Xóa một tag chỉ xóa quan hệ của tag đó; không xóa các event từng gắn tag.

4.2. Functional requirements cốt lõi

Event CRUD

- Tạo, xem chi tiết, sửa, xóa sự kiện.
- Trường: title, start time, end time, location, description, tags; color là tùy chọn.
- Title bắt buộc; `EndAt > StartAt`.
- Location và description có thể rỗng.
- Cho phép event trùng/chồng thời gian; không tự ghi đè event khác.
- Xóa phải có bước xác nhận.

Calendar

- MVP có Month view và Week view.
- Có Previous / Next / Today.
- Event xuất hiện đúng ngày và được sắp xếp theo start time.
- Chuyển tháng/tuần phải tải đúng khoảng ngày, bao gồm ngày thuộc tháng trước/sau trong month grid.

Tags

- Tạo, sửa tên/màu, xóa tag.
- Gắn nhiều tag vào một event.
- Lọc riêng `tagA`, `tagB` và `Chưa có tag`.

Search / Filter

- Tìm theo title, description, location.
- Lọc theo khoảng ngày.

- Kết hợp keyword + date range + tag filter.
- Có Clear filters và trạng thái no-result.
- Nên kiểm thử tiếng Việt có dấu và ký tự đặc biệt. Lưu ý SQLite `NOCASE` không phải bộ so sánh Unicode đầy đủ; nếu nhóm muốn tìm kiếm không phân biệt dấu tiếng Việt, hãy thống nhất và triển khai một hàm normalize riêng thay vì mặc định cho rằng `COLLATE NOCASE` đã xử lý mọi trường hợp.

Ngoài MVP / chỉ làm khi còn thời gian: all-day event, recurrence, reminder/notification, import/export, đồng bộ cloud, nhiều tài khoản.

---

5. SQLite và quy tắc truy cập dữ liệu

5.1. Quy tắc chung

- Chỉ sử dụng `Microsoft.Data.Sqlite`; không dùng Entity Framework Core.
- Tách SQL khỏi View và ViewModel; SQL nằm trong `Repositories`.
- Mỗi operation tạo connection qua `SqliteConnectionFactory`, sau đó `Dispose` bằng `using`.
- Bật `PRAGMA foreign_keys = ON` cho mỗi connection.
- Mọi input người dùng phải truyền qua `SqliteParameter`; không nối chuỗi input vào câu SQL.
- Dùng transaction khi một thao tác gồm nhiều câu lệnh liên quan, ví dụ lưu event và các tag của nó.
- Không commit file database runtime vào Git. Commit `schema.sql` và script seed nếu cần.

5.2. Schema MVP đề xuất

```
CREATE TABLE IF NOT EXISTS events (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    title           TEXT NOT NULL
                    CHECK (length(trim(title)) > 0),
    description     TEXT NULL,
    location        TEXT NULL,
    start_at_utc    TEXT NOT NULL,
    end_at_utc      TEXT NOT NULL,
    color_hex       TEXT NULL,
    created_at_utc  TEXT NOT NULL,
    updated_at_utc  TEXT NOT NULL,
    CHECK (end_at_utc > start_at_utc)
);

CREATE TABLE IF NOT EXISTS tags (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    name      TEXT NOT NULL COLLATE NOCASE UNIQUE,
    color_hex TEXT NULL
);

CREATE TABLE IF NOT EXISTS event_tags (
    event_id INTEGER NOT NULL,
    tag_id   INTEGER NOT NULL,
    PRIMARY KEY (event_id, tag_id),
    FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id)   REFERENCES tags(id)   ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_events_start_end
    ON events(start_at_utc, end_at_utc);

CREATE INDEX IF NOT EXISTS idx_event_tags_tag
    ON event_tags(tag_id);
```

`schema.sql` trong project phải là nguồn chuẩn. Nếu chỉnh schema, cập nhật file SQL và thực hiện thay đổi qua PR; không tự ý đổi schema chỉ trong máy của một thành viên.

5.3. DateTime

- Dùng `DateTimeOffset` trong C# để biểu diễn thời điểm.
- Lưu `start_at_utc`, `end_at_utc`, `created_at_utc`, `updated_at_utc` dưới dạng ISO 8601 UTC nhất quán, ví dụ `DateTimeOffset.UtcDateTime.ToString("O")`.
- Khi hiển thị, chuyển UTC về local time.
- Khi người dùng chọn khoảng ngày local, tạo mốc đầu ngày và mốc đầu ngày kế tiếp, sau đó chuyển sang UTC trước khi query.
- Với lọc khoảng ngày, có thể dùng điều kiện overlap: `event.start_at_utc < @toExclusiveUtc AND event.end_at_utc > @fromUtc`. Nhờ vậy event kéo dài qua nhiều ngày vẫn được tìm thấy.

5.4. Lưu file DB

Đặt database trong thư mục user-writable, ví dụ dưới `Environment.SpecialFolder.LocalApplicationData`, không đặt cạnh file `.exe` hoặc trong repository. Database initializer phải tạo DB/schema khi chạy lần đầu và không xóa dữ liệu khi ứng dụng khởi động lại.

5.5. Đa luồng với Microsoft.Data.Sqlite — cần chú ý

SQLite không hỗ trợ asynchronous I/O; các API ADO.NET có hậu tố `Async` trong `Microsoft.Data.Sqlite` **thực thi đồng bộ**. Vì thế:

- Không viết `ExecuteReaderAsync()` chỉ để ghi điểm "async" nếu operation thực tế vẫn chạy đồng bộ.
- Với CRUD nhỏ và DB local, giữ thao tác ngắn, query có index phù hợp và tránh tải dữ liệu không cần thiết.
- Nếu có tác vụ thật sự dài (ví dụ backup/export hoặc import nhiều dữ liệu), có thể chạy **hàm SQLite đồng bộ** trong `Task.Run` để chuyển phần việc khỏi UI thread.
- Tạo connection bên trong tác vụ nền; không dùng chung một `SqliteConnection` giữa nhiều thread.
- Không cập nhật WPF control trực tiếp từ worker thread. Trả kết quả về ViewModel/UI thread hoặc marshal qua `Dispatcher`.
- Chỉ thêm progress/busy state khi có operation đủ lâu để người dùng cần phản hồi.

Nguồn tham khảo: Microsoft.Data.Sqlite — Async limitations.

---

6. WPF-UI, ResourceDictionary và Template

- Khai báo theme/control dictionaries của WPF-UI trong `App.xaml` theo hướng dẫn của version NuGet nhóm đang cài.
- Style dùng chung để trong `Resources/Styles`; DataTemplate dùng chung để trong `Resources/Templates`.
- Đặt resource key có nghĩa, không lặp một style tương tự ở nhiều View.
- Nếu hỗ trợ đổi theme runtime, ưu tiên `DynamicResource` cho màu/brush cần đổi theo theme.
- Dùng `DataTemplate` cho event chip, tag chip và item trong danh sách thay vì tạo UI bằng code-behind.
- Icon/ảnh để trong `Resources/Assets`; ghi nguồn/license nếu lấy từ bên ngoài.
- Dùng control của WPF-UI nhất quán; không trộn nhiều hệ style chỉ để dùng thật nhiều loại control.

Tài liệu: WPF-UI GitHub · Application Themes.

---

7. Git workflow cho nhóm

7.1. Quy tắc branch

Chỉ giữ `main` là nhánh ổn định. Mỗi task làm trên branch riêng, tạo từ `main`.

Loại	Naming	Ví dụ
Tính năng mới	`feature/<task-id>-<short-name>`	`feature/T054-calendar-month-grid`
Sửa lỗi	`fix/<task-id>-<short-name>`	`fix/T062-invalid-end-time`
Tài liệu	`docs/<task-id>-<short-name>`	`docs/T107-readme-setup`
Công việc kỹ thuật	`chore/<task-id>-<short-name>`	`chore/T016-add-packages`

Dấu `/` trong `feature/calendar-month-grid` **chỉ là quy ước đặt tên**, không phải branch lồng trong branch.

Không dùng tên thành viên làm tên branch. Branch nên nói rõ task đang làm; tên người phụ trách đã được ghi ở Task Tracker/PR.

7.2. Quy tắc commit

Cấu trúc:

```
<type>(<scope>): <mô tả ngắn ở dạng mệnh lệnh>
```

Type	Dùng khi	Ví dụ
`feat`	Thêm chức năng	`feat(calendar): add month grid`
`fix`	Sửa lỗi	`fix(event): validate end time`
`refactor`	Tái cấu trúc, không đổi behavior	`refactor(repository): split event queries`
`ui`	Chỉnh giao diện/style/template	`ui(calendar): refine event chip style`
`test`	Thêm/sửa kiểm thử	`test(search): cover untagged filter`
`docs`	Tài liệu	`docs(readme): add setup steps`
`chore`	Package/config/tooling	`chore(deps): add WPF-UI package`

- Một commit nên mô tả một thay đổi có ý nghĩa.
- Không dùng message chung chung như `update`, `fix`, `final`, `test`.

- Không commit `bin/`, `obj/`, `.vs/`, file database runtime hoặc file cấu hình cá nhân.

7.3. Quy trình tạo branch và push

```
# 1. Cập nhật main ở local
git switch main
git pull origin main

# 2. Tạo branch từ main mới nhất
git switch -c feature/T054-calendar-month-grid

# 3. Code, kiểm tra thay đổi
git status
git diff

# 4. Commit
git add src/MnemoCalendar/Views/CalendarView.xaml
git add src/MnemoCalendar/ViewModels/CalendarViewModel.cs
git commit -m "feat(calendar): add month grid"

# 5. Push branch lần đầu
git push -u origin feature/T054-calendar-month-grid
```

Sau đó tạo Pull Request trên GitHub:

- **Base:** `main`
- **Compare:** branch feature/fix/docs của task
- Mô tả: mục tiêu, thay đổi chính, cách test, ảnh/video nếu có chỉnh UI.
- Link task ID (ví dụ `T054`) hoặc GitHub Issue.
- Chờ ít nhất một thành viên review và approve trước khi merge.

7.4. PR checklist

```
[ ] PR giải quyết một task hoặc nhóm task liên quan.
[ ] Đã pull/merge main mới nhất vào branch.
[ ] Project build thành công.
[ ] Đã test luồng bị ảnh hưởng và regression cơ bản.
[ ] Không có SQL hoặc business logic trong code-behind.
[ ] Query dùng parameters; transaction/FK đúng nếu có thay đổi DB.
[ ] Binding, command và validation hoạt động.
[ ] Đã cập nhật schema/docs nếu có thay đổi.
[ ] Không commit bin/obj/.vs/database runtime hoặc file cá nhân.
[ ] Có ít nhất một reviewer approve.
```

7.5. Cập nhật branch khi main có commit mới

Với nhóm mới học Git, ưu tiên `merge origin/main` vào branch feature thay vì rebase branch đã push dùng chung. Cách này dễ hiểu và không phải force-push.

```
git switch feature/T054-calendar-month-grid
git fetch origin
git merge origin/main

# Nếu có conflict: sửa file, sau đó
git status
git add <file-da-giai-quyet>
git commit -m "merge: sync main into feature/T054-calendar-month-grid"
git push
```

7.6. Xử lý conflict

Khi Git báo conflict:

1. Chạy `git status` để biết file nào đang conflict.
2. Mở file trong VS Code; tìm các marker:
   - `<<<<<<< HEAD`: nội dung từ branch hiện tại.
   - `=======`: ranh giới hai phiên bản.
   - `>>>>>>> ...`: nội dung từ branch được merge.
3. Đọc cả hai thay đổi và **tự quyết định nội dung cuối cùng**. Không xóa marker một cách máy móc hoặc luôn chọn `ours` / `theirs`.
4. Xóa toàn bộ marker, giữ lại phần code hợp nhất có chủ đích.
5. Build project và test các luồng liên quan.
6. `git add` file đã giải quyết, commit merge và push branch.
7. Báo cho người tạo PR/reviewer biết conflict đã được xử lý.

Các file cần phối hợp trước khi sửa: `MainWindow.xaml`, `App.xaml`, `.csproj`, `schema.sql`, ResourceDictionary dùng chung và các ViewModel trung tâm. Hai người cùng sửa một file XAML lớn rất dễ tạo conflict khó ghép.

Không làm:

- Không push trực tiếp vào `main`.
- Không dùng `git push --force` để “chữa” conflict.
- Không xóa file hoặc chọn toàn bộ phiên bản của một phía nếu chưa hiểu thay đổi của phía kia.
- Không merge PR khi reviewer chưa approve hoặc project chưa build/test.

7.7. `.gitignore` gợi ý cho WPF

```
# Visual Studio
.vs/
*.suo
*.user
*.userosscache
*.sln.docstates

# Build output
[Bb]in/
[Oo]bj/

# Logs
*.log

# Local SQLite runtime files
*.db
*.db-shm
*.db-wal
*.sqlite
*.sqlite3

# OS/editor
.DS_Store
Thumbs.db
```

Nếu cần dữ liệu mẫu, hãy tạo bằng `seed.sql` hoặc code seed riêng; không commit file DB runtime của thành viên.

---

8. Definition of Done — khi nào một task được xem là hoàn thành?

Chỉ tick ☑ trong Task Tracker khi:

- ☐ Đã đáp ứng acceptance criteria của task.
- ☐ Project build thành công.
- ☐ Đã test trường hợp chính và ít nhất một trường hợp lỗi/boundary phù hợp.
- ☐ UI binding/command cập nhật đúng; không đưa nghiệp vụ vào code-behind.
- ☐ Dữ liệu SQLite được lưu/đọc đúng nếu task có liên quan đến dữ liệu.
- ☐ Đã commit và push lên đúng branch.
- ☐ PR có mô tả và đã được reviewer approve.
- ☐ Đã merge vào `main` theo quy tắc của nhóm.

Sau khi merge, cập nhật Task Tracker và ghi chú/evidence nếu task có điểm cần trình bày khi demo.

---

9. Tài liệu tham khảo chính thức

- CommunityToolkit.Mvvm — ObservableProperty
- CommunityToolkit.Mvvm — RelayCommand
- Microsoft.Data.Sqlite — Async limitations
- WPF-UI — GitHub repository
- WPF-UI — Application Themes